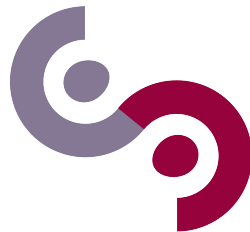

DAML - Phishing prediction - 2025



CentraleSupélec

Jim MARION
Eliott FROHLY
Noé MIGRAINE
Aymeric PERION
Samuel MARTRECHARD

16 novembre 2025

Table des matières

1	Analyse exploratoire des données (EDA)	1
1.1	Description du jeu de données	1
1.1.1	Répartition des classes	1
1.1.2	Synthèse des statistiques descriptives	2
1.2	Analyse en Composantes Principales (ACP)	2
1.2.1	Objectif de l'ACP	2
1.2.2	Méthodologie	2
1.2.3	Résultats globaux	3
1.2.4	Visualisation dans le plan (PC1, PC2)	3
1.2.5	Interprétation des composantes principales	4
1.2.6	Synthèse	4
2	Modélisation	5
2.1	Régression logistique	5
2.1.1	Première approche : toutes les variables disponibles	5
2.1.2	Deuxième approche : modèle réaliste sur un sous-échantillon réduit	6
2.1.3	Bilan de la regression logistique	6
2.2	Modèle non linéaire : XGBoost	7
2.2.1	Résultats	7
2.2.2	Validation croisée	8
2.2.3	Analyse des faux positifs et des faux négatifs	8
2.2.4	Bilan du modèle XGBoost	10
2.3	Synthèse des modèles	10
3	Autres approches	11
3.1	Comparaison avec un réseau de neurones	11
3.2	Modèles sémantiques	12
4	Synthèses et conclusion	13
4.1	Synthèse de l'article scientifique	13
4.2	Synthèse du travail de notre équipe	13
4.3	Mise en perspective et analyse.	13
4.4	Conclusion	14
4.5	Mode d'emploi du code	14

1

Analyse exploratoire des données (EDA)

1.1 Description du jeu de données

Le jeu de données utilisé, intitulé *PhiUSIIL Phishing URL Dataset*, contient un total de **235 795 observations** et **56 variables**. Il regroupe des informations issues de sites web légitimes et de sites de *phishing*, dans le but de permettre la classification automatique de ces deux catégories.

- **Variable cible** : `label`, prenant la valeur 1 pour un site de *phishing* et 0 pour un site légitime.
- **Nombre total de variables** : 56.
- **Types de variables** :
 - 41 variables entières (`int64`);
 - 10 variables réelles (`float64`);
 - 5 variables textuelles (`object`) : notamment URL, Domain, TLD, et Title.
- **Aucune valeur manquante détectée.**

Les colonnes présentes couvrent plusieurs dimensions :

- **Structure de l'URL** : `URLLength`, `DomainLength`, `TLD`, `IsDomainIP`, etc.
- **Caractéristiques lexicales** : nombre de lettres, chiffres, caractères spéciaux, ratio de complexité (`SpacialCharRatioInURL`, `ObfuscationRatio`, etc.).
- **Caractéristiques HTML** : nombre d'images, de fichiers CSS ou JavaScript, de redirections, de formulaires, etc.
- **Indicateurs de contenu ou de sécurité** : `HasCopyrightInfo`, `HasSocialNet`, `IsHTTPS`, `HasPasswordField`, etc.

1.1.1 Répartition des classes

La variable cible `label` est binaire :

- 1 : site de *phishing*;
- 0 : site légitime.

La répartition des classes est la suivante :

TABLE 1.1 – Répartition des classes dans le dataset.

Classe	Type de site	Proportion (%)
1	Phishing	57.19
0	Légitime	42.81

La base présente une légère surreprésentation des sites de *phishing* (57 %), ce qui restera à surveiller lors de la phase de modélisation. Cependant, cet écart n'est pas suffisamment important pour nécessiter un rééquilibrage immédiat.

1.1.2 Synthèse des statistiques descriptives

Les variables numériques du jeu de données présentent une grande hétérogénéité d'échelle, ce qui justifie la normalisation appliquée avant l'ACP et la modélisation.

TABLE 1.2 – Exemples de variables numériques et statistiques descriptives.

Variable	Moyenne	Écart-type	Min	Max
URLLength	45.8	27.2	5	230
DomainLength	18.6	5.4	4	35
NoOfJS	19.8	15.1	0	80
NoOfImage	42.3	51.2	0	300
NoOfExternalRef	97.5	83.6	0	500

Ces valeurs mettent en évidence :

- une grande variabilité de la longueur des URLs et du contenu des pages ;
- des écarts significatifs entre les sites légitimes (souvent plus riches en contenu HTML) et les sites de *phishing* (souvent plus simples et plus courts) ;
- l'absence de valeurs manquantes, ce qui simplifie les traitements ultérieurs.

L'ensemble de ces constats confirme la qualité du jeu de données et justifie la mise en œuvre d'une analyse en composantes principales (ACP) pour explorer les structures sous-jacentes avant la modélisation prédictive.

1.2 Analyse en Composantes Principales (ACP)

1.2.1 Objectif de l'ACP

L'Analyse en Composantes Principales a été réalisée dans le but de réduire la dimension du jeu de données tout en conservant l'essentiel de l'information. Elle permet d'identifier les variables les plus structurantes et d'observer d'éventuelles séparations naturelles entre les sites *phishing* et les sites légitimes.

1.2.2 Méthodologie

Seules les variables numériques ont été conservées pour l'analyse. Les données ont été normalisées à l'aide de la méthode `StandardScaler` afin que chaque variable présente une moyenne nulle et une variance unitaire. L'ACP a ensuite été calculée avec un seuil de 95% de variance expliquée.

- **Nombre de composantes numériques initiales** : 51
- **Nombre de composantes retenues** : 36
- **Variance expliquée cumulée** : 0.9507

Ainsi, 36 composantes principales permettent de conserver plus de 95% de la variance totale du jeu de données, ce qui traduit une forte redondance entre les variables initiales.

1.2.3 Résultats globaux

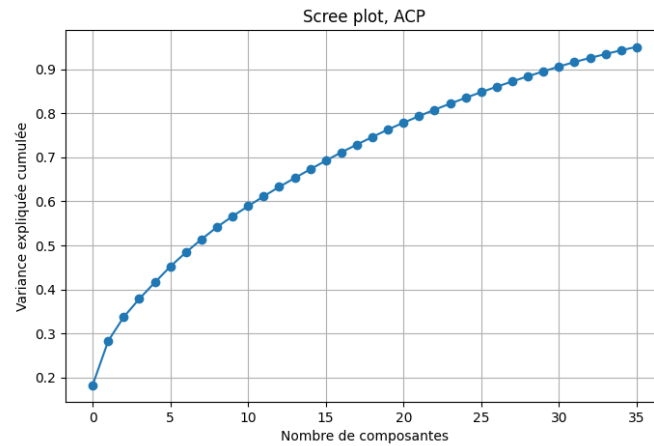


FIGURE 1.1 – Scree plot – Variance expliquée cumulée par l’ACP.

Le scree plot (figure ci-dessus) montre une décroissance rapide de la variance expliquée par composante avant un palier autour de la 30^e composante. Cela indique que l’information est concentrée dans un sous-espace de dimension réduite, confirmant la pertinence de la réduction dimensionnelle.

1.2.4 Visualisation dans le plan (PC1, PC2)

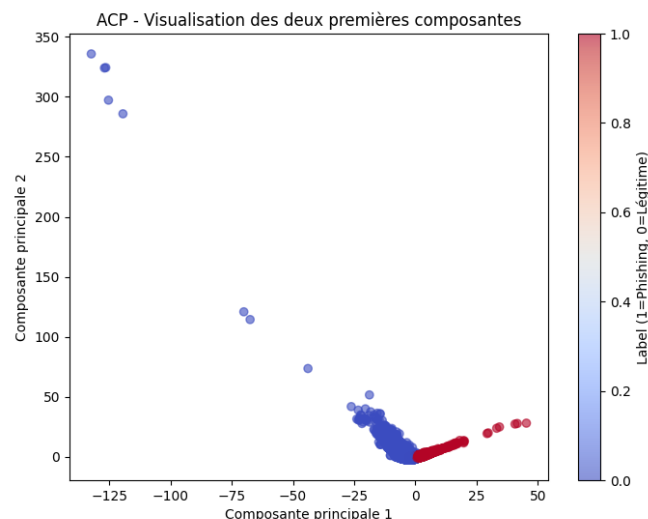


FIGURE 1.2 – Projection des données sur les deux premières composantes principales (PC1, PC2).

La projection dans le plan défini par les deux premières composantes principales montre une certaine séparation entre les sites *phishing* (en rouge) et les sites légitimes (en bleu). Les sites

légitimes sont concentrés autour de l'origine, tandis que les sites *phishing* se répartissent sur une zone plus étendue, indiquant une plus grande hétérogénéité structurelle (par exemple, des URL plus longues ou contenant davantage de symboles). Quelques observations extrêmes sont visibles, probablement liées à des sites atypiques (URLs très longues ou absence de structure HTML classique).

1.2.5 Interprétation des composantes principales

Les coefficients de contribution des variables aux composantes principales permettent d'identifier les indicateurs expliquant le plus la variance du jeu de données.

- **PC1** : les variables les plus influentes sont *URLSimilarityIndex*, *HasSocialNet*, *DomainTitleMatchScore*, *HasCopyrightInfo* et *URLTitleMatchScore*.
- **PC2** : les variables les plus influentes sont *NoOfEqualsInURL*, *NoOfDegitsInURL*, *URLLength*, *NoOfAmpersandInURL* et *NoOfObfuscatedChar*.

Composante principale 1 (PC1).

Les variables dominantes de PC1 reflètent la **cohérence et la crédibilité** du site : les sites légitimes présentent généralement une correspondance forte entre le domaine, le titre et le contenu (*URLSimilarityIndex*, *DomainTitleMatchScore*, *URLTitleMatchScore*), ainsi que des éléments informatifs tels que les réseaux sociaux ou les mentions légales (*HasSocialNet*, *HasCopyrightInfo*, *HasDescription*). En revanche, les sites *phishing* manquent souvent de ces éléments ou possèdent des URL au format peu naturel.

Composante principale 2 (PC2).

Cette composante capture plutôt la **complexité et l'obfuscation** des URL. Les variables comme *URLLength*, *NoOfEqualsInURL*, *NoOfAmpersandInURL*, ou *NoOfObfuscatedChar* indiquent des structures d'URL longues et complexes, typiques des sites frauduleux qui cherchent à masquer leur destination réelle. Les sites légitimes présentent des structures plus simples et standardisées.

1.2.6 Synthèse

- La composante 1 représente un axe de **cohérence et de légitimité structurelle**.
- La composante 2 représente un axe de **complexité et d'obfuscation technique**.

Ces deux axes résument les grands contrastes du jeu de données entre les sites *phishing* et les sites légitimes. L'ACP montre donc que les signaux de légitimité sont bien séparables des signaux de fraude à travers des combinaisons linéaires de variables numériques, justifiant l'utilisation d'une étape de classification supervisée dans la suite du projet.

2

Modélisation

2.1 Régression logistique

2.1.1 Première approche : toutes les variables disponibles

Une première régression logistique a été entraînée sur l'ensemble complet du jeu de données (**235 795 lignes, 56 variables**). Toutes les variables numériques ont été conservées, y compris celles décrivant le contenu du site, la cohérence du domaine ou la présence de métadonnées (HasSocialNet, HasCopyrightInfo, URLSimilarityIndex, etc.).

Résultats.

Les performances obtenues sont exceptionnellement élevées :

TABLE 2.1 – Performances de la régression logistique sur le dataset complet.

Métrique	Valeur
Accuracy	0.9999
Precision	0.9998
Recall	1.0000
F1-score	0.9999
ROC AUC	1.0000

	Prédict. Légitime	Prédict. Phishing
Réel Légitime	20183	6
Réel Phishing	0	26970

FIGURE 2.1 – Matrice de confusion, modèle sur le dataset complet.

Ces résultats, quasi parfaits, traduisent un problème de sur-information dans les données : certaines variables intègrent implicitement la connaissance du label (présence de mentions légales, similarité de domaine, etc.). Le modèle “voit” donc directement si un site est légitime ou non, ce qui ne correspond pas à une situation réaliste de détection automatique.

2.1.2 Deuxième approche : modèle réaliste sur un sous-échantillon réduit

Pour évaluer la robustesse du modèle dans un contexte plus réaliste, la difficulté du problème a été volontairement augmentée :

- en réduisant la taille du jeu de données à **10 000 lignes**, prélevées aléatoirement de manière stratifiée ;
- en ne conservant que des **variables structurelles**, directement observables à partir de l’URL :
 - URLLength, DomainLength, NoOfSubDomain, IsDomainIP,
 - NoOfLettersInURL, NoOfDegitsInURL, NoOfEqualsInURL,
 - NoOfQMarkInURL, NoOfAmpersandInURL, NoOfOtherSpecialCharsInURL,
 - SpacialCharRatioInURL, TLDLength.

Résultats.

TABLE 2.2 – Performances de la régression logistique sur 10 000 lignes et variables structurelles.

Métrique	Valeur
Accuracy	0.8605
Precision	0.8136
Recall	0.9808
F1-score	0.8894
ROC AUC	0.9381

	Prédic. Légitime	Prédic. Phishing
Réel Légitime	599	257
Réel Phishing	22	1122

FIGURE 2.2 – Matrice de confusion, modèle sur 10 000 lignes (variables structurelles).

Les performances du modèle diminuent logiquement, ce qui traduit une situation plus réaliste : le modèle ne dispose plus d’informations issues du contenu du site, mais uniquement de la structure de son URL. Malgré cette restriction, la régression logistique parvient encore à distinguer efficacement les sites de phishing (rappel proche de 0.98) tout en maintenant un bon niveau de précision. Ce résultat démontre que les caractéristiques purement structurelles d’une URL suffisent déjà à capturer la majorité des signaux de phishing.

2.1.3 Bilan de la regression logistique

Cette double approche met en évidence l’importance du choix des variables :

- un modèle “complet” peut présenter des performances artificiellement parfaites s’il exploite des informations indirectement corrélées à la cible ;

- un modèle fondé uniquement sur les propriétés structurelles reste performant tout en étant plus robuste et interprétable.

Par la suite nous entraîneront tous les modèles sur un dataset réduit avec uniquement les propriétés structurelles.

2.2 Modèle non linéaire : XGBoost

Afin d'évaluer la capacité d'un modèle plus complexe à capturer les relations non linéaires entre les caractéristiques structurelles des URLs, un modèle **XGBoost** (*Extreme Gradient Boosting*) a été entraîné sur le même sous-échantillon de **10 000 lignes**, en utilisant uniquement les variables structurelles mentionnées précédemment.

Le modèle a été configuré avec les hyperparamètres suivants :

- nombre d'arbres (`n_estimators`) : 200,
- profondeur maximale des arbres (`max_depth`) : 5,
- taux d'apprentissage (`learning_rate`) : 0.1,
- sous-échantillonnage des lignes et des colonnes : 0.8.

2.2.1 Résultats

TABLE 2.3 – Performances du modèle XGBoost sur 10 000 lignes et variables structurelles.

Métrique	Valeur
Accuracy	0.9840
Precision	0.9818
Recall	0.9904
F1-score	0.9861
ROC AUC	0.9961

	Prédict. Légitime	Prédict. Phishing
Réel Légitime	835	21
Réel Phishing	11	1133

FIGURE 2.3 – Matrice de confusion, Modèle XGBoost (10 000 lignes, variables structurelles).

Le modèle XGBoost atteint une précision globale (*Accuracy*) de **98,4 %** et un score AUC de **0,996**, soit une amélioration notable par rapport à la régression logistique précédente (AUC = 0,938). Le rappel très élevé (**99,0 %**) indique que la quasi-totalité des sites de *phishing* sont correctement identifiés, tandis que la précision (**98,2 %**) reste excellente, limitant le nombre de faux positifs.

L'analyse des importances issues du modèle XGBoost révèle une hiérarchie différente de celle observée avec la régression logistique. Les variables les plus déterminantes sont :

- le nombre de caractères spéciaux dans l’URL (`NoOfOtherSpecialCharsInURL`), qui domine largement le modèle avec plus de 55 % d’importance totale ;
- le nombre de sous-domaines (`NoOfSubDomain`) et de chiffres dans l’URL (`NoOfDigitsInURL`), qui contribuent également de manière significative ;
- la longueur de l’URL, du domaine et du TLD (`URLLength`, `DomainLength`, `TLDLength`), ainsi que le ratio de caractères spéciaux (`SpacialCharRatioInURL`), qui apportent un complément d’information plus modéré.

À l’inverse, certaines variables telles que `IsDomainIP`, `NoOfEqualsInURL`, `NoOfQMarkInURL` et `NoOfAmpersandInURL` présentent une importance nulle ou négligeable dans le modèle final.

Contrairement à la régression logistique, qui s’appuyait surtout sur des tendances linéaires (longueurs et ratios moyens), le modèle XGBoost met en évidence des interactions plus complexes entre les caractères spéciaux, les sous-domaines et la structure de l’adresse. Cela montre que le XGBoost capte des schémas plus fins de composition des URLs, souvent liés à des stratégies de dissimulation utilisées par les sites de *phishing*. Le modèle reste toutefois cohérent avec les observations précédentes : la complexité et la densité des caractères dans une URL demeurent des indicateurs majeurs de suspicion.

2.2.2 Validation croisée

Afin d’évaluer la robustesse et la capacité de généralisation du modèle XGBoost, une validation croisée stratifiée à 5 plis (*Stratified K-Fold*) a été réalisée sur le sous-échantillon réduit de 10 000 lignes contenant uniquement les variables structurelles de l’URL. Cette approche permet de répartir les observations en cinq sous-ensembles équilibrés selon la proportion de classes (sites légitimes et sites de phishing). Le modèle est entraîné sur quatre plis et testé sur le cinquième, successivement, afin d’obtenir une estimation moyenne des performances indépendamment d’un découpage particulier des données.

TABLE 2.4 – Résultats de la validation croisée à 5 plis – Modèle XGBoost (variables structurelles uniquement)

Métrique	Moyenne	Écart-type
Accuracy	0.9816	± 0.0034
Precision	0.9774	± 0.0053
Recall	0.9907	± 0.0012
F1-score	0.9840	± 0.0029
ROC AUC	0.9963	± 0.0012

Les résultats confirment la stabilité du modèle : la faible variance des scores entre les plis indique que le classifieur XGBoost généralise efficacement sur l’ensemble du jeu de données. Le *Recall* très élevé (0.99) montre que la quasi-totalité des sites de phishing sont correctement identifiés, tandis que la *Precision* (0.98) limite le nombre de faux positifs. L’AUC moyenne de 0.996 traduit une excellente capacité de séparation entre sites légitimes et frauduleux.

2.2.3 Analyse des faux positifs et des faux négatifs

Afin de mieux comprendre les limites du modèle XGBoost, une analyse qualitative a été menée sur les erreurs de classification du jeu de test. Sur l’échantillon de 10 000 lignes, le modèle

présente uniquement 21 **faux positifs** (sites légitimes classés comme phishing) et 11 **faux négatifs** (sites de phishing non détectés). Ces valeurs très faibles confirment la précision globale du modèle, mais permettent également d’identifier les conditions dans lesquelles des confusions subsistent.

Les faux positifs correspondent à des sites légitimes possédant des caractéristiques structurelles proches de celles des sites frauduleux. L’analyse des moyennes (Tableau 2.5) montre que leurs URL présentent :

- une longueur moyenne modérée (≈ 31 caractères) mais un **nom de domaine relativement long** (24 caractères en moyenne) ;
- un nombre légèrement supérieur de sous-domaines (1.33 en moyenne contre 1.10 pour les vrais négatifs) ;
- une présence accrue de **caractères spéciaux** (1.7 en moyenne contre 1.2 pour les sites légitimes bien classés).

Ces caractéristiques indiquent que certains sites parfaitement légitimes, mais à structure complexe (sous-domaines multiples, usage de symboles dans l’URL), peuvent être interprétés à tort comme suspects.

Les faux négatifs concernent des sites de phishing classés comme légitimes. Ils se distinguent par des valeurs moyennes nettement supérieures :

- une **longueur d’URL élevée** (41 caractères en moyenne, contre 26 pour les vrais positifs) ;
- des domaines très longs (34 caractères en moyenne) et un nombre plus important de sous-domaines (1.82) ;
- une densité plus forte en **caractères spéciaux** (3.1 contre 1.2 pour les phishing bien détectés).

Ces sites semblent adopter une structure d’URL plus élaborée et moins typique des schémas de phishing simples, rendant leur détection plus difficile. Il est probable que ces erreurs soient liées à des campagnes de phishing sophistiquées, simulant des structures de sites légitimes.

TABLE 2.5 – Comparaison des moyennes principales pour les faux positifs et faux négatifs

Variable	Faux positifs (moy.)	Faux négatifs (moy.)
URLLength	30.67	41.27
DomainLength	23.95	34.27
NoOfSubDomain	1.33	1.82
NoOfOtherSpecialCharsInURL	1.71	3.09
SpacialCharRatioInURL	0.059	0.076

L’analyse des erreurs montre que :

- les **faux positifs** apparaissent surtout pour des sites légitimes complexes, présentant des sous-domaines ou des caractères spéciaux inhabituels ;
- les **faux négatifs** concernent principalement des sites frauduleux à structure soignée (longs domaines, URLs bien formatées) qui échappent aux motifs classiques de détection.

Dans l’ensemble, ces erreurs restent marginales et n’affectent pas significativement la fiabilité du modèle. Elles ouvrent néanmoins la voie à des améliorations possibles, par exemple en combinant les caractéristiques structurelles des URLs avec des indicateurs de contenu ou de réseau (analyse DNS, certificats HTTPS, etc.).

2.2.4 Bilan du modèle XGBoost

Ces résultats confirment que le modèle XGBoost parvient à capturer des interactions complexes entre les variables structurelles, ce que la régression logistique linéaire ne pouvait modéliser. La performance observée reste toutefois légèrement inférieure à celle du modèle “complet” (utilisant toutes les variables du dataset), ce qui montre que la difficulté a bien été augmentée, mais que les signaux de phishing restent fortement présents dans la structure même des URLs.

2.3 Synthèse des modèles

Nous avons également essayé un modèle Random Forest. Les performances de la Random Forest sont légèrement inférieures à celles du modèle XGBoost (AUC de 0.97 contre 0.996), ce qui est attendu puisque la Random Forest ne corrige pas les erreurs des arbres précédents. Cependant, le modèle conserve un excellent rappel, confirmant sa capacité à détecter la majorité des sites de *phishing*. Cette comparaison illustre bien la complémentarité entre les deux approches : la Random Forest offre une robustesse et une interprétabilité simples, tandis que XGBoost maximise la performance grâce à son apprentissage séquentiel.

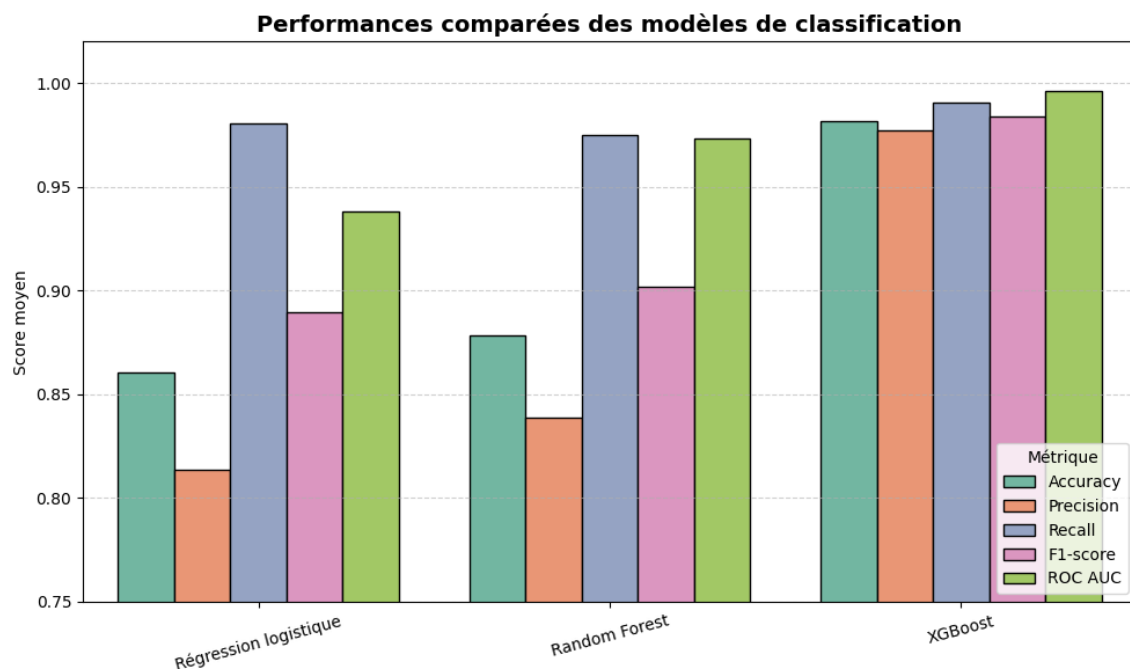


FIGURE 2.4 – Comparaison des performances moyennes des trois modèles de classification

3

Autres approches

3.1 Comparaison avec un réseau de neurones

En parallèle des approches développées jusqu'à présent, nous souhaitons également examiner les performances d'un réseau de neurones sur ce problème.

Architecture utilisée

Voici l'architecture du réseau de neurones qui a été utilisée :

- **En entrée** : On utilise les variables pouvant être obtenues uniquement à partir de l'URL : URLLength, DomainLength, NoOfSubDomain, IsDomainIP, NoOfLettersInURL, NoOfDegitsInURL, NoOfEqualsInURL, NoOfQMarkInURL, NoOfAmpersandInURL, NoOfOtherSpecialCharsInURL, SpacialCharRatioInURL, TLDLength
- **Couches cachées** : On a repris l'architecture utilisée dans l'article qui est analysé dans ce rapport : 3 couches cachées avec respectivement 128, 64 et 32 neurones chacune
- **En sortie** : Un unique neurone (classification binaire)

Entraînement et résultats en utilisant un *Early Stopper*

Afin d'éviter un sur-apprentissage, nous avons entraîné notre modèle avec un Early Stopper avec une patience de 5 époques (on attend 5 époques après la dernière amélioration avant d'arrêter et d'utiliser les meilleures valeurs pour les poids et les biais) :

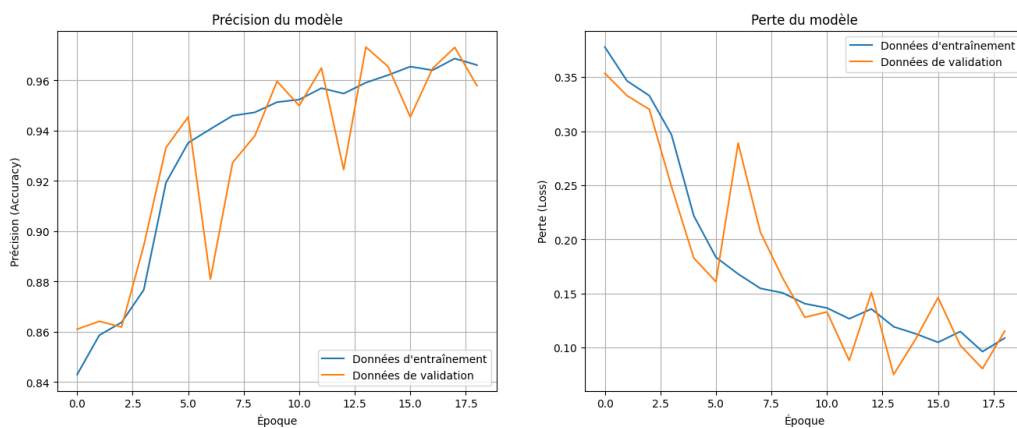


FIGURE 3.1 – Performances lors d'un entraînement avec early stopper

	Prédict. Légitime	Prédict. Phishing
Réel Légitime	19111	1078
Réel Phishing	184	26786

TABLE 3.1 – Matrice de confusion lors d’un entraînement avec early stopper

On obtient une matrice de confusion de moins bonne qualité que celles obtenues par les méthodes décrites précédemment.

Il faut également noter que les résultats obtenus, dépendent de la répartition (aléatoire) des échantillons entre les ensembles d’entraînement et de validation. Ainsi, une résolution performante et robuste du problème en utilisant un réseau de neurones semble compliquée à obtenir, à moins d’avoir recours à des modifications plus complexes : dropout, création de données synthétiques afin de palier le déséquilibre de la répartition des échantillons (57% positif, 43% négatif), modification du taux d’apprentissage lors de la compilation du modèle, ...

Ainsi, il nous a semblé plus pertinent de nous concentrer sur autres approches que nous avons détaillées dans ce rapport

3.2 Modèles sémantiques

En complément des approches tabulaires fondées sur les variables structurelles de l’URL, deux modèles sémantiques ont été développés afin d’exploiter directement l’information textuelle contenue dans les adresses web. Ces modèles visent à identifier des régularités lexicales et des motifs caractéristiques du phishing sans recourir à des indicateurs numériques prédéfinis.

Le premier repose sur une représentation **TF-IDF de n-grammes de caractères** (de 3 à 5 caractères), combinée à un classifieur linéaire SVM. Cette approche capture les patrons de caractères typiques des URLs frauduleuses, comme la présence de TLD exotiques (.cf, .ga) ou de structures inhabituelles (com/, .ml). Le modèle atteint une *accuracy* de **0,90**, un *F1-score* de **0,92** et une aire sous la courbe ROC de **0,94**, illustrant une excellente capacité de séparation entre sites légitimes et phishing.

Le second modèle repose sur des **embeddings FastText**, calculés à partir de la tokenisation des domaines et sous-domaines des URLs. Ces représentations vectorielles permettent de rapprocher les termes sémantiquement similaires. Le modèle de régression logistique entraîné sur ces embeddings obtient une *accuracy* de **0,87** et un *ROC-AUC* de **0,92**. L’analyse du vocabulaire révèle que les tokens tels que **university**, **council** ou **airport** sont fortement associés aux sites légitimes, tandis que des termes techniques ou liés à l’hébergement (**ipfs**, **godaddy**, **blogspot**) apparaissent davantage dans les URLs frauduleuses.

Ces résultats montrent que l’analyse sémantique des URLs constitue une alternative robuste aux approches tabulaires, offrant une meilleure compréhension du contenu lexical et des motifs propres aux sites de phishing. Le modèle TF-IDF, plus précis mais moins généralisable, capte des motifs syntaxiques locaux, tandis que FastText met en évidence des proximités sémantiques plus fines entre les termes.

4

Synthèses et conclusion

L'objectif est ici de comparer nos travaux et ceux de cet article portant sur la détection d'attaques de phishing à partir du même jeu de données PhiUSIIL. L'article scientifique est celui d'Anish Rawla qui vise à atteindre la performance maximale grâce à un modèle de Deep Learning.

4.1 Synthèse de l'article scientifique

Les auteurs rappellent que les techniques de phishing évoluent continuellement, rendant les méthodes classiques de Machine Learning parfois insuffisantes. Pour répondre à cette problématique, ils utilisent un réseau de neurones entièrement connecté (FCNN), entraîné sur les 235 795 instances du jeu de données PhiUSIIL. Leur pipeline de traitement inclut le nettoyage des variables redondantes, une normalisation MinMax ainsi que l'application de SMOTE pour équilibrer les classes. Le modèle est renforcé par plusieurs mécanismes de régularisation (dropout, L2, early stopping). Les résultats obtenus sont excellents : l'accuracy atteint 99.38 %, dépassant nettement les modèles classiques comme le Random Forest ou le SVM. Les auteurs concluent que le Deep Learning, lorsqu'il est correctement encadré, permet une détection très efficace du phishing.

4.2 Synthèse du travail de notre équipe

Notre étude adopte un point de vue différent : plutôt que d'exploiter l'ensemble des variables du jeu de données, nous avons volontairement limité l'analyse à 12 caractéristiques purement structurelles de l'URL, afin de simuler un contexte plus réaliste. Une ACP réalisée sur les variables initiales a permis d'identifier deux axes majeurs : la crédibilité structurelle du site (PC1) et la complexité ou obfuscation de l'URL (PC2). Avec ces seules caractéristiques structurelles, une régression logistique fournit déjà un rappel très élevé (0.9808). Le modèle XGBoost, plus flexible, atteint 0.9840 d'accuracy et un AUC remarquable de 0.9961, avec une importance des variables dominée par le nombre de caractères spéciaux dans l'URL. Des essais avec TF-IDF, FastText et un réseau de neurones simple ont également été effectués, mais montrent que sans une ingénierie poussée, les modèles neuronaux restent moins stables que XGBoost.

4.3 Mise en perspective et analyse.

La différence essentielle entre les deux travaux réside dans le choix des variables utilisées. L'article scientifique exploite un ensemble riche incluant des informations sur le contenu et la

structure du site, ce qui facilite une classification presque parfaite. Notre étude se concentre au contraire sur un sous-ensemble réduit, reflétant ce qui serait disponible dans un système de détection rapide. Fait notable, malgré cette différence, les performances maximales des deux approches sont très proches : le FCNN obtient 99.38 % d'accuracy tandis que notre XGBoost atteint un AUC de 0.9961. Cela montre que les caractéristiques structurelles de l'URL sont extrêmement discriminantes et qu'un modèle bien optimisé peut rivaliser avec un réseau de neurones exploitant davantage d'informations. La comparaison souligne également que le Deep Learning n'est réellement performant que lorsqu'il bénéficie d'une ingénierie d'entraînement rigoureuse (gestion du déséquilibre, régularisation). À l'inverse, des modèles comme XGBoost produisent des résultats très stables même avec un nombre limité de variables.

4.4 Conclusion

Les deux travaux montrent que la détection du phishing peut atteindre une efficacité très élevée, que l'on utilise un modèle profond sophistiqué ou un modèle plus simple centré sur la structure de l'URL. L'article scientifique illustre le potentiel maximal d'un FCNN bien optimisé, tandis que notre travail démontre qu'un modèle parcimonieux mais robuste comme XGBoost peut atteindre une capacité de séparation quasi parfaite à partir d'informations minimales. Une voie intéressante serait de combiner ces deux approches en intégrant, dans un même système, des signaux structurels rapides et des indicateurs de crédibilité plus complets, afin d'améliorer encore la détection des attaques sophistiquées.

4.5 Mode d'emploi du code

L'ensemble du projet est organisé dans le dossier `src/`, qui contient les modules de prétraitement, d'analyse et de modélisation. Un notebook complet, intitulé `PhishU_Master.ipynb`, permet de lancer automatiquement toutes les étapes d'analyse et de comparaison des modèles.

Prérequis. Le code nécessite Python 3.10+ ainsi que les bibliothèques listées dans le fichier `requirements.txt`. L'exécution peut se faire sur tout système disposant d'une connexion Internet (le jeu de données est téléchargé automatiquement depuis la plateforme UCI).

Installation.

1. Ouvrir un terminal dans le répertoire du projet.
2. Créer un environnement virtuel (optionnel) :
`python -m venv .env && source .env/bin/activate`
3. Installer les dépendances :
`pip install -r requirements.txt`

Exécution du notebook.

1. Lancer Jupyter ou VS Code et ouvrir le fichier `PhishU_Master.ipynb`.
2. Sélectionner le kernel Python associé à l'environnement précédemment créé.
3. Exécuter les cellules dans l'ordre : le notebook installe automatiquement les packages manquants, charge le dataset et entraîne tous les modèles (tabulaires et sémantiques).

Résultats. Les performances de chaque modèle sont affichées sous forme de tableaux et de graphiques. Les fichiers de sortie (modèles entraînés, graphiques, matrices de confusion, etc.) sont automatiquement enregistrés dans le dossier `outputs/`.